

Verified Heuristic Memory Management for Long-Context Language Models

Dynamic Span Eligibility, Parent–Child Memory Trees, and Multi-Step Verification

Magnus Theodore ✉ 
Independent Researcher

Abstract

Long-context language models are increasingly used as persistent assistants, code agents, and research systems, but their memory mechanisms are still often treated as passive compression or flat retrieval. A sliding window preserves recency rather than importance. A generic summary may delete small but decisive tokens such as negations, thresholds, version markers, file names, or design constraints. Embedding-based retrieval can return semantically similar but stale or superseded fragments. These failures suggest that long-context memory needs a promotion mechanism: a span should not become long-term memory merely because it was observed; it should earn memory status through salience, recurrence, utility, consistency, and evidence.

This paper proposes *Verified Heuristic Memory Management* (VHM), a lightweight theoretical framework for memory governance in long-context language models. The framework assigns dynamic eligibility weights to candidate token spans, ranks a bounded number of candidates, verifies them through a multi-step protocol, and stores promoted memories in a parent–child memory tree. Parent nodes represent topics, claims, and constraints, while child nodes preserve raw spans, evidence, and examples. A promoted high-level memory must maintain an evidence path to raw text. The system also records supersession edges so that newer decisions can invalidate older ones without erasing historical evidence.

The central mechanism is *Multi-Step Salience Verification* (MSV). A candidate span is first proposed by cheap heuristic or teacher-derived weighting. It then passes through local salience checking, parent attachment, sibling consistency checking, optional lazy replay or ablation, and finally a promotion decision into Candidate, Shadow, Active, Pinned, Stale, Superseded, or Rejected state. The paper formulates the memory promotion problem and derives structural properties of the framework: bounded online cost under top- K verification, optimality of top- K ordering under equal-cost estimated utility, reduction of false promotion probability by staged verification, preservation of evidence-path invariants, supersession-aware retrieval, and convergence of dynamic weights toward observed utility under stable feedback. The goal is not to replace attention or retrieval, but to add a verifiable memory lifecycle above them.

2012 ACM Subject Classification Computing methodologies → Natural language processing; Information systems → Information retrieval; Theory of computation → Data structures design and analysis

Keywords and phrases Long-context language models, memory management, heuristic memory, salience verification, context compression, retrieval, evidence-backed memory, dynamic token weights

Digital Object Identifier Not assigned

Category Technical proposal and theoretical framework

Funding No external funding

1 Introduction

Long-context language models have made it possible to place very large amounts of text into a model’s context window. Yet a larger context window does not by itself solve the memory problem. In long technical dialogues, research workflows, and software engineering sessions, the important information is often sparse, brittle, and nonlocal. A later question may depend on a variable name introduced thousands of tokens earlier, a design choice that superseded a previous decision, or a negative constraint such as “do not reduce the confirmation depth.” Such information is easy to damage by summarization and easy to miss by pure semantic retrieval.



© Magnus Theodore;
licensed under [Creative Commons License CC-BY 4.0](https://creativecommons.org/licenses/by/4.0/)
Technical Manuscript.
Editors: Magnus Theodore;
Article No. 1; pp. 1:1–1:8

TECHNICAL ANALYSIS

Independent research manuscript in long-context memory systems
Self-contained proceedings-style technical draft.

1:2 Verified Heuristic Memory

The usual solutions address pieces of the problem. Sliding-window attention preserves recent material, but recency and importance are different signals. Prompt compression and summary memory reduce length, but compression may erase polarity, exact symbols, and evidence. Retrieval-augmented generation retrieves semantically similar chunks, but semantic similarity is not the same as current validity. A stale design note may be more similar to a query than the later note that replaced it. Related techniques include retrieval-augmented generation [2], prompt compression [3, 4], gist-token compression [5], streaming attention with sink tokens [6], and OS-style external memory for language agents [7]. The missing layer is not another fixed attention pattern, but a memory governance mechanism.

This paper proposes to treat memory formation as a promotion process. A token span begins as raw context. It becomes a candidate only if a cheap salience mechanism gives it sufficient weight. It becomes shadow memory only after local verification. It becomes active memory only when attached to a suitable parent and checked against siblings. It becomes pinned memory only after repeated use, replay utility, or strong constraint value. Conversely, a memory can become stale or superseded when newer evidence contradicts it. The result is a lifecycle:

Raw → Candidate → Shadow → Active → Pinned,

with side exits to Rejected, Stale, and Superseded.

We call the resulting framework *Verified Heuristic Memory Management* (VHM). It is heuristic because it uses cheap, approximate signals to propose candidates: hard symbols, numbers, negations, constraint words, repetition in prompts, and prior teacher salience. It is verified because those candidates do not become trusted long-term memory until they pass a multi-step verification protocol. The verification protocol is intentionally bounded: the whole context is scanned cheaply, but only a fixed number of top candidates receive expensive checks.

The framework is independent of a particular base language model. It can be placed above a long-context transformer, a retrieval-augmented system, a KV-cache manager, an agent memory, or a prompt compression system. Its purpose is to decide what deserves to be remembered and how that memory should be indexed, invalidated, and justified.

Contributions. This manuscript makes four conceptual contributions.

- It formulates the *Memory Promotion Problem*: under bounded memory and retrieval budgets, decide which candidate spans deserve long-term memory status.
- It introduces *Dynamic Span Eligibility*, where initial teacher or heuristic salience is updated by recurrence, usage, replay utility, constraint risk, conflict, staleness, and redundancy.
- It proposes a *Parent-Child Verified Memory Tree*: parent nodes manage topics, claims, and constraints, while child nodes preserve evidence and raw spans. Promoted high-level memories must maintain evidence paths.
- It defines *Multi-Step Salience Verification* (MSV), a bounded promotion protocol that reduces false promotion, supports supersession, and separates historical relevance from current validity.

2 Motivation and Positioning

A long-context system can fail even when the required information remains inside the context. The failure may occur because the model attends to a stale decision, because a summary collapsed a prohibition into a vague topic label, or because an index retrieved a semantically close but invalidated chunk. These failures are especially common in technical conversations, where small symbols and logical polarity carry large weight.

Consider a design discussion in which the system first considers allowing an aggressive search mode to reduce re-search depth, and later decides that the aggressive mode must not reduce fail-high confirmation depth. A flat summary may record only that aggressive mode affects re-search depth. A semantic retriever may return both the old and the new design notes without distinguishing which is current. A verified memory manager should instead store the later statement as an active constraint, mark the older alternative as superseded, and retain evidence spans for both.

The proposed framework differs from ordinary prompt compression in its objective. A compression system asks which tokens can be removed while preserving immediate task performance. VHM asks which spans have earned long-term memory status. It also differs from ordinary retrieval: an embedding index can retrieve relevance, but it does not by itself represent support, contradiction, supersession, or current validity. VHM therefore adds structure around retrieval: state, evidence paths, parent attachment, and verification history.

3 Problem Formulation

Let a context stream be a sequence of tokens

$$C_t = (x_1, x_2, \dots, x_t).$$

A span is an interval

$$s = (i, j), \quad 1 \leq i \leq j \leq t.$$

Let $\mathcal{S}_t$ denote the set of candidate spans generated from C_t . A memory manager selects a set of memory nodes $\mathcal{M}_t$ subject to a budget B , where the budget may count stored tokens, retrieval tokens, raw evidence links, or KV-cache entries.

Future queries are sampled from an unknown distribution $\mathcal{Q}$. The ideal utility of retaining a span s is

$$U(s) = \mathbb{E}_{q \sim \mathcal{Q}} [\Delta(s, q)],$$

where $\Delta(s, q)$ is the improvement in answer quality when evidence from s is available for query q . This quantity is not directly observable. The system maintains a dynamic estimate $W_t(s)$, called the eligibility weight.

► **Problem 1 (Memory Promotion Problem).** Given a context stream, a bounded memory budget B , an unknown future query distribution $\mathcal{Q}$, and noisy evidence about span utility, maintain a memory set $\mathcal{M}_t$ and a retrieval policy that maximize expected future utility while minimizing false promotion and stale recall.

The problem is deliberately not only a compression problem. A compressed summary may be short, but unsupported or stale. A promoted memory must satisfy stronger requirements: it should have evidence, a parent location, a state, and a policy for invalidation.

4 Dynamic Span Eligibility

A span should not be assigned a fixed importance score at extraction time. Its importance depends on later use. A variable name that appears once may be irrelevant; the same variable name repeated in prompts, code changes, and error traces becomes a stable anchor. A constraint that is never used may remain shadow memory; a constraint that prevents later errors should be promoted.

We define the dynamic eligibility weight as

$$W_t(s) = \alpha W_{\text{init}}(s) + \beta R_t(s) + \gamma A_t(s) + \delta M_t(s) + \eta C(s) - \lambda F_t(s) - \mu D_t(s) - \nu H_t(s).$$

Here W_{init} is a teacher-derived or heuristic prior; R_t is recurrence in prompts and work artifacts; A_t is successful usage after retrieval; M_t is replay or ablation utility; C is constraint, hard-symbol, negation, or compression-risk score; F_t is freshness penalty due to staleness or supersession; D_t is redundancy with a parent claim; and H_t is conflict penalty.

The initial weight is a prior rather than a verdict. It may be assigned by a small salience extractor, a large teacher model, or hand-written rules. Subsequent updates transform the prior into a posterior-like estimate of usefulness. In the simplest case, usage feedback can be tracked by an exponentially weighted moving average:

$$W_{t+1}(s) = (1 - \rho)W_t(s) + \rho r_t(s),$$

where $r_t(s)$ is positive if the memory helped, zero if unused, and negative if it caused stale or conflicting recall.

1:4 Verified Heuristic Memory

Heuristic signals. A low-cost implementation can score spans by the presence of code symbols, file names, version numbers, thresholds, named entities, explicit constraints, negations, repeated anchors, and user corrections. Exact anchor lookup can be implemented with multi-pattern string matching such as Aho–Corasick [1]. The goal is not to decide final memory status, but to order candidates for verification. In this sense token weighting is analogous to move ordering in search: it decides what receives expensive verification first.

5 Heuristic Memory Index

A memory system needs an index that is richer than a flat embedding table. Semantic similarity is useful, but many memories are best retrieved by exact anchors, parent paths, states, and constraints. We therefore define a *Heuristic Memory Index* as a set of partial indexes over memory nodes:

$$I = (I_{\text{anchor}}, I_{\text{entity}}, I_{\text{constraint}}, I_{\text{parent}}, I_{\text{state}}, I_{\text{time}}, I_{\text{conflict}}, I_{\text{sem}}).$$

At query time, anchors and constraints first identify candidate parent nodes. The retriever then explores bounded active or pinned children under those parents, optionally mixing in a small number of semantic candidates. Stale and superseded nodes are not used as current answer evidence unless the query explicitly asks for history.

A typical retrieval score is

$$\text{score}(v, q) = a(v, q) + c(v, q) + p(v, q) + \sigma(v) + W_t(v) + r(v, q) - f(v) - h(v),$$

where a is anchor match, c is constraint match, p is parent-path match, σ is state bonus, r is semantic relevance, f is stale or superseded penalty, and h is conflict penalty.

The purpose of the index is to avoid global top- k retrieval over all chunks whenever possible. First locate the relevant parent region of the memory tree, then retrieve bounded children and evidence paths.

6 Parent–Child Verified Memory Tree

A memory node has a type, state, payload, anchors, evidence links, and weights. The main node types are

RawSpan, Evidence, Claim, Constraint, Topic, Root.

The main states are

Candidate, Shadow, Active, Pinned, Stale, Superseded, Rejected.

Edges record more than containment. They may represent

Supports, Refines, Contradicts, Supersedes, ExampleOf, ConstraintOf.

The central invariant is evidence preservation.

► **Definition 1 (Evidence path).** An evidence path for a memory node v is a directed path from v to at least one Evidence or RawSpan node through support or example edges.

► **Definition 2 (Promoted memory).** A memory node is promoted if its state is Active or Pinned and its type is Claim or Constraint.

► **Definition 3 (Evidence Path Invariant).** Every promoted Claim or Constraint node must have at least one evidence path to raw context.

This invariant separates verified memory from unsupported summary. A high-level conclusion is allowed only if the system can point back to the material that supports it. The claim may still be wrong if the verifier was wrong, but the system remains auditable.

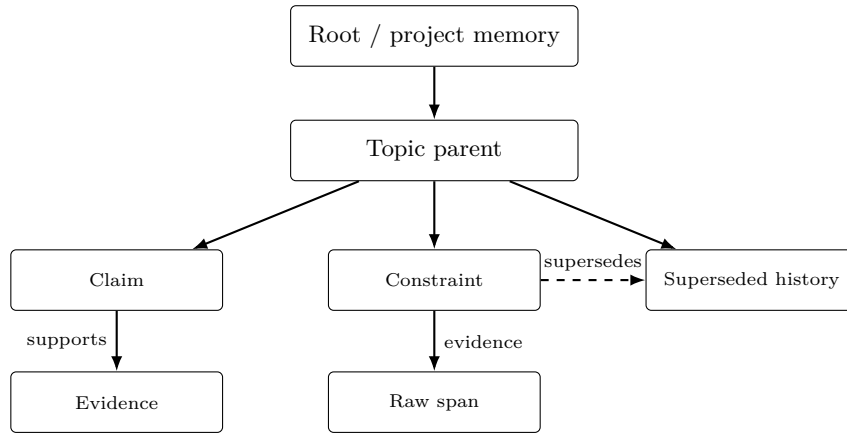


Figure 1: Parent-child verified memory tree. Parent nodes route topics and manage children. Promoted claims and constraints retain evidence paths to raw spans. Supersession edges preserve old information without allowing it to dominate current retrieval.

Algorithm 1 Bounded Multi-Step Saliency Verification

Require: candidate spans S , memory tree T , cap K

- 1: compute eligibility weights $W_t(s)$ for $s \in S$
 - 2: let $Q \leftarrow \text{TopK}(S, W_t, K)$
 - 3: **for** each $s \in Q$ **do**
 - 4: **if** not locally salient **then**
 - 5: mark s as Rejected
 - 6: continue
 - 7: **end if**
 - 8: choose candidate parent nodes using the heuristic memory index
 - 9: attach s to the best parent or create a new shadow parent
 - 10: compare against bounded siblings for support, contradiction, or supersession
 - 11: **if** evidence is weak or conflicting **then**
 - 12: store as Shadow or Stale
 - 13: **else if** utility is confirmed or constraint risk is high **then**
 - 14: promote to Active or Pinned
 - 15: **else**
 - 16: compress or keep as Shadow
 - 17: **end if**
 - 18: **end for**
-

7 Multi-Step Saliency Verification

MSV is a bounded protocol for memory promotion. The system does not run the full protocol on every span. It first ranks candidates by dynamic eligibility and then verifies only a fixed number of top spans.

The stages are deliberately conservative. A span can be useful without becoming pinned. A span can be historically relevant but superseded. A summary can exist without being allowed to answer current queries. These distinctions are absent from flat memory systems.

8 Theoretical Properties

The properties below are structural. They do not claim empirical superiority over every long-context method. They show that, under explicit assumptions, the proposed framework has desirable invariants and bounded cost.

► **Theorem 1 (Bounded online cost).** *Assume token weighting is a linear scan over n tokens, anchor matching takes $O(n+m)$ time for m matches, the MSV cap is K , and the number of*

1:6 Verified Heuristic Memory

candidate parents per span is bounded by P . Then each online update costs

$$O(n+m+K\log K+KP).$$

If K and P are constants, the cost is $O(n+m)$.

Proof. The token scorer scans the input once, giving $O(n)$. Anchor or automaton matching contributes $O(n+m)$. Candidate extraction is bounded by the number of matches and high-weight spans. Selecting the top K candidates costs at most $O(K\log K)$ after bounded candidate filtering, or can be implemented by a fixed-size heap during scanning. MSV compares each selected span with at most P candidate parents, hence $O(KP)$. Summing terms gives the bound. When K and P are fixed system constants, all non-scanning terms are constant with respect to n , yielding $O(n+m)$. $\square$

► **Theorem 2 (Top- K ordering under equal cost).** *Suppose every candidate span has equal storage and verification cost and $W(s)$ is the estimated future utility of span s . Then selecting the K spans of largest $W(s)$ maximizes estimated total utility among all size- K selections.*

Proof. Let M be an optimal size- K set that is not the top- K set. Then there exist $a \notin M$ and $b \in M$ with $W(a) > W(b)$. Replacing b by a increases the total estimated utility, contradicting optimality of M . Hence the optimal set consists of the top- K spans by weight. $\square$

► **Theorem 3 (False promotion reduction).** *Let F be the event that a candidate span is a false memory candidate. Suppose promotion requires passing verification stages $V_1, \dots, V_r$, and for each stage*

$$\Pr[V_i | F, V_1, \dots, V_{i-1}] \leq \varepsilon_i.$$

Then

$$\Pr[\text{promote} | F] \leq \prod_{i=1}^r \varepsilon_i.$$

Proof. Promotion occurs only when all stages pass. By the chain rule,

$$\Pr[V_1 \wedge \dots \wedge V_r | F] = \prod_{i=1}^r \Pr[V_i | F, V_1, \dots, V_{i-1}].$$

Each factor is at most ε_i , so the product bound follows. $\square$

► **Theorem 4 (Evidence path preservation).** *If every promotion of a Claim or Constraint to Active or Pinned requires an evidence path, and update operations do not delete all raw evidence supporting a promoted node, then the Evidence Path Invariant is preserved.*

Proof. Initially the memory tree is empty, so the invariant holds. Adding raw spans or evidence nodes cannot create unsupported promoted claims. Adding candidate or shadow claims does not violate the invariant because they are not promoted. A promotion operation is allowed only when an evidence path exists. Stale and superseded transitions only lower current validity and do not create new promoted claims. Compression or merging is permitted only if at least one evidence link is retained. Therefore every operation preserves the invariant, and induction over the update sequence proves the claim. $\square$

► **Theorem 5 (Supersession-aware retrieval).** *Suppose a node B is marked as superseding node A , with A set to Superseded and B set to Active or Pinned. If current-answer retrieval excludes Superseded nodes unless the query explicitly asks for historical information, then A is not used as current answer evidence.*

Proof. The retrieval candidate set for current answers contains only nodes in active current states, such as Active and Pinned. Since A is in state Superseded, it is not included in the candidate set. Therefore it cannot be selected as current answer evidence under the stated policy. $\square$

► **Theorem 6 (Dynamic weight tracking).** *Let usage feedback $r_t(s)$ for a span s be drawn from a stationary distribution with expectation μ . If the dynamic weight update is*

$$W_{t+1}(s) = (1 - \rho)W_t(s) + \rho r_t(s), \quad 0 < \rho \leq 1,$$

then $\mathbb{E}[W_t(s)] \rightarrow \mu$.

Proof. Taking expectations gives

$$\mathbb{E}[W_{t+1}] = (1 - \rho)\mathbb{E}[W_t] + \rho\mu.$$

Let $e_t = \mathbb{E}[W_t] - \mu$. Then $e_{t+1} = (1 - \rho)e_t$, so $e_t = (1 - \rho)^t e_0$. Since $0 < \rho \leq 1$, the factor tends to zero, and $\mathbb{E}[W_t] \rightarrow \mu$. $\square$

9 Discussion

VHM is not a replacement for transformer attention. It is a governance layer above context, retrieval, compression, and cache management. A long-context model may still attend over a recent window. A retrieval system may still retrieve semantically related passages. A prompt compressor may still reduce context length. VHM adds a lifecycle: proposed memories are verified, attached, indexed, promoted, invalidated, and justified by evidence.

The framework is especially suited to domains where exact small tokens matter. In code assistance, a function name, file path, build flag, or negative constraint may dominate future correctness. In research assistance, a later design decision may supersede earlier hypotheses. In agent settings, task state must be distinguished from conversation history. In all of these cases, memory quality depends less on total stored text and more on preserving the right spans with the right validity state.

Relation to compression. Prompt compression attempts to keep enough information for a current or near-future task. VHM can use compression, but it refuses to let compression erase evidence. A compressed parent claim is useful only if it retains links to raw child spans.

Relation to retrieval. Retrieval ranks relevance. VHM ranks eligibility and validity. A stale memory may be highly relevant but should not be used as current evidence. A superseded node may still be useful when the user asks how a decision evolved.

Relation to human memory tools. The parent-child tree resembles note hierarchies, issue trackers, and version control. The difference is that the system explicitly records promotion state and verification history. A memory is not merely a note; it is a node with a claim, state, support, and invalidation policy.

10 Limitations and Future Work

This manuscript is a theoretical proposal and does not report empirical results. The quality of the framework depends on the quality of the salience scorer and verifier. Parent attachment may fail, especially when anchors are ambiguous. Replay or ablation utility can be expensive and may need to be lazy. Dynamic weights can overfit to recent usage if the update rate is too large. Evidence-path invariants make memories auditable but do not guarantee that the evidence was interpreted correctly.

Future work should implement a small prototype, construct long technical memory benchmarks, train a dedicated salience extractor from human or large-model teachers, and evaluate constraint recall, stale error rate, evidence hit rate, and token-budget efficiency. Another direction is integration with KV-cache routing: high-weight spans could be pinned in cache, compressed into learned memory tokens, or retrieved through the heuristic memory index.

11 Conclusion

Long-context systems should not merely remember more. They should manage memory more carefully. This paper proposed Verified Heuristic Memory Management, a framework in which spans receive dynamic eligibility weights, pass through multi-step verification, and are stored in a parent-child memory tree with evidence paths and supersession states. The framework reframes memory as a lifecycle of candidate, shadow, active, pinned, stale, superseded, and rejected states. Its central principle is simple:

no promoted memory without evidence.

Memory is therefore not passive context retention, but verified promotion under bounded budget.

References

- [1] Alfred V. Aho and Margaret J. Corasick. Efficient string matching: An aid to bibliographic search. *Communications of the ACM*, 18(6):333–340, 1975.
- [2] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.
- [3] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLM-Lingua: Compressing prompts for accelerated inference of large language models. arXiv:2310.05736, 2023.
- [4] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLM-Lingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. arXiv:2403.12968, 2024.
- [5] Jesse Mu, Xiang Lisa Li, and Noah D. Goodman. Learning to compress prompts with gist tokens. In *NeurIPS*, 2023.
- [6] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. arXiv:2309.17453, 2023.
- [7] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. arXiv:2310.08560, 2023.